프롬프트 엔지니어링 (기본)

프롬프트 엔지니어링 개요

1. 프롬프트 엔지니어링 개요

프롬프트 엔지니어링은?

언어 모델(LM)을 효과적으로 활용하기 위해 프롬프트를 개발하고 최적화하는 분야이다.이 기술은 대규모 언어 모델(LLM)의 기능과 한계를 이해하는 데 도움을 준다.

연구자들은 프롬프트 엔지니어링을 통해 LLM의 성능을 향상시키고, 개발자들은 이를 바탕으로 LLM과의 인터페이스를 설계한다. 프롬프트 엔지니어링은 LLM의 안전성을 개선하고, 도메인 지식과 외부 도구를 활용하여 새로운 기능을 추가하는 데 중요한 역할을 한다.

1. 프롬프트 엔지니어링 개요

프롬프트 엔지니어링은 단순히 프롬프트를 만드는 것 이상으로, LLM과 상호작용하며 기능을 이해하고 개선하는 기술이다.

이를 통해 LLM의 안전성을 높이고, 새로운 기능을 추가할 수 있다.

https://www.promptingguide.ai/kr

프롬프트 구성 요소

- **지시(Instruction)** : 모델이 수행할 특정 작업 또는 지시
- **문맥(Context)** : 더 나은 응답을 위해 모델을 조종할 수 있는 외부 정보나 추가 문맥
- **입력 데이터(Input data)** : 응답 받고자 하는 입력이나 질문
- **출력 지시자(Output indicator)** : 출력의 유형이나 형식

구성 요소 예

- **지시(Instruction):** 모델이 수행해야 할 구체적인 작업을 명시합니다. 예를 들어, "다음 주제에 대해 5줄짜리 시를 써봐: 고독"과 같이 말이죠.
- **문맥(Context):** 모델이 지시를 더 잘 이해하고, 더욱 정확한 결과를 내도록 돕는 추가적인 정보입니다. 예를 들어, "고독"이라는 주제에 대해 특정한 시대나 스타일을 지정하거나, 특정한 단어를 사용하도록 요청할 수 있습니다.
- **입력 데이터(Input Data):** 모델이 처리해야 할 초기 데이터입니다. 예를 들어, 번역을 요청할 때 원문을 입력하거나, 요약을 요청할 때 원문을 입력하는 것이죠.
- **출력 지시자(Output Indicator):** 모델이 생성해야 할 출력의 형식이나 스타일을 지정합니다. 예를 들어, "짧고 간결한 문장으로 요약해줘" 또는 "재미있는 이야기 형식으로 설명해줘"와 같이 말할 수 있습니다.

파이썬 소스 코드에서 프롬프트 구성 요소 설정 예시

```python
# 프롬프트 구성 요소 설정
instruction = "다음 글을 요약해 주세요:"
context = "이 글은 기후 변화가 해양 생물에 미치는 영향에 대한 과학적 논문에서 발췌한 것입니다."
input_data = "연구에 따르면, 해수 온도의 상승이 산호초 개체수의 급격한 감소를 초래했다고 합니다."
output_directive = "요약은 간결하게 작성하고, 일반 대중이 이해할 수 있는 형태로 작성해 주세요."

# 프롬프트 결합
prompt = f"{instruction}\n\n문맥: {context}\n\n텍스트: {input_data}\n\n{output_directive}"
```

코드 설명:

1. **지시**: "다음 글을 요약해 주세요:" - 모델이 수행할 작업을 지시합니다.
2. **문맥**: "이 글은 기후 변화가 해양 생물에 미치는 영향에 대한 과학적 논문에서 발췌한 것입니다." - 모델이 더 잘 이해하도록 문맥 정보를 제공합니다.
3. **입력 데이터**: "연구에 따르면, 해수 온도의 상승이 산호초 개체수의 급격한 감소를 초래했다고 합니다." - 모델이 처리할 텍스트입니다.
4. **출력 지시자**: "요약은 간결하게 작성하고, 일반 대중이 이해할 수 있는 형태로 작성해 주세요." - 출력 형식에 대한 지시입니다.

API 호출 결과(OpenAI)

```python
# OpenAI API 호출
client = OpenAI(api_key=API_KEY)

response = client.chat.completions.create(
  model="gpt-4o-mini",
  messages=[ {"role": "user", "content": prompt}]
)

# 응답 출력
print(response.choices[0].message.content)
```

```
연구 결과, 해수 온도가 상승함에 따라 산호초 개체수가 급격히 감소하고 있다는 사실이 밝혀졌습니다.
```

프롬프트에 구성 요소가 모두 필요한 것은 아님.

API 호출(Gemini)

```python
import vertexai
from vertexai.generative_models import GenerativeModel

# 1. Vertex AI 초기화 (프로젝트 ID와 리전 설정)
vertexai.init(project="your-project-id", location="us-central1")

# 2. 모델 로드 (가성비 좋은 Flash 모델 선택)
model = GenerativeModel("gemini-1.5-flash-001")

# 3. 프롬프트 작성 및 요청
prompt = "인공지능 개발자가 되기 위해 공부해야 할 3가지를 요약해줘."
response = model.generate_content(prompt)

# 4. 결과 출력
print(response.text)
```

```python
from vertexai.generative_models import GenerationConfig

config = GenerationConfig(
    temperature=0.7,  # 적당한 창의성 부여
    max_output_tokens=500  # 답변 길이 제한
)

response = model.generate_content(
    "창의적인 마케팅 슬로건 5개 만들어줘",
    generation_config=config
)
```

프롬프트에 구성 요소가 모두 필요한 것은 아님.

LLM 설정 매개 변수

(1) Temperature: 낮추면 더 결정적이고 사실적인 응답을, 높이면 창의적이고 다양한 출력을 얻습니다.

(2) Top P: 확률 질량을 구성하는 토큰만 사용해 응답의 다양성을 조절합니다. 온도와 함께 사용하지만, 둘 중 하나만 변경하는 것이 좋습니다.

(3) Max Length(Maximum Token, max_output_tokens) : 생성되는 응답의 길이를 제어합니다

LLM 설정 매개변수

(3) Stop Sequences : 특정 문자열에서 응답 생성을 중단시킴

(4) Frequency Penalty: 이미 사용된 단어의 반복을 줄입니다. 값이 클수록 반복이 더 억제됩니다.

(5) Presence Penalty: 반복되는 모든 단어에 동일한 페널티를 적용해 구문의 반복을 방지합니다.값이 클수록 새로운 내용을 더 많이 도입하려고 시도합니다.

Temperature의 작동 방식

온도(temperature)값의 범위는 0에서 무한대이지만 일반적으로 0.5 ~ 1.0 사이의 값을 주로 사용 한다. Temperature 값을 지정하지 않은 경우 기본 값은 1.0이다

값이 낮을 때 (예: 0.2):

모델이 가장 높은 확률을 가진 단어를 선택할 가능성이 높아집니다. 따라서 결과가 매우 결정적(deterministic)이고, 예측 가능한 텍스트가 생성됩니다.예를 들어, 뉴스 요약이나 사실 기반 질의 응답과 같이 명확하고 정확한 응답이 필요한 작업에서 유용합니다.

모델의 "창의성"이 줄어들고, 응답이 반복적이거나 단조로울 수 있습니다.

Temperature의 작동 방식

값이 높을 때 (예: 1.0이상):

모델이 확률이 낮은 단어들도 선택할 가능성이 높아져, 결과가 다양하고 예측 불가능하게 됩니다.

이로 인해 더 창의적이고 독특한 텍스트가 생성될 수 있습니다.

예를 들어, 시나 소설 생성, 창의적인 글쓰기 작업 등에서 활용될 수 있습니다.

그러나 값이 너무 높으면 텍스트가 무의미하거나 비논리적일 수 있습니다.

Temperature 설정 예시:

Temperature = 0.0 : 모델이 확률적으로 가장 높은 단어만 선택하게 되어, 완전히 결정적인 결과를 제공합니다. 이 경우 사실상 "확률적 선택"이 사라지며, 항상 같은 입력에 같은 출력이 생성됩니다.

Temperature = 0.7 : 모델이 확률에 따라 다소 다양하지만 여전히 신뢰할 수 있는 텍스트를 생성합니다. 이는 보통의 대화나 글쓰기 작업에서 자주 사용되는 값입니다.

Temperature = 1.5 : 모델이 매우 다양하고 창의적인 텍스트를 생성하지만, 일관성과 논리성이 떨어질 수 있습니다.

Top P의 작동 방식: Top P 값을 지정하지 않은 경우 기본 값은 1.0이다

- **확률 질량 누적 합**: Top P는 모델이 생성할 다음 단어를 선택할 때, 해당 단어들이 가질 확률의 누적 합이 P(%)를 넘는 순간까지 후보군에 포함시킵니다. 예를 들어, Top P가 0.9로 설정되어 있다면, 모델은 상위 90%의 확률을 차지하는 단어들만 후보로 고려하게 됩니다.
- **다양성 조절**: Top P의 값을 낮게 설정하면(예: 0.1), 확률이 가장 높은 몇 가지 단어만 고려되어 결과가 매우 결정적(deterministic)이 됩니다. 반면, 값을 높게 설정하면(예: 0.9 이상), 더 많은 단어가 후보군에 포함되어 보다 다양한 결과를 생성할 수 있습니다.

예시:

- **Top P = 1.0**: 모델은 가능한 모든 단어 후보군을 고려합니다. 이는 전혀 제약 없이 모델이 가장 가능성이 높은 단어부터 낮은 단어까지 모두 검토하여 선택할 수 있음을 의미합니다.
- **Top P = 0.5**: 모델은 가장 가능성이 높은 단어들 중에서만 후보를 선택합니다. 예를 들어, 상위 50%의 확률 질량을 차지하는 단어들만 고려하게 되어, 더 집중적이고 예측 가능한 결과를 얻게 됩니다.

생성에 미치는 영향

Top-p 값이 높을수록(1.0에 가까울수록) 더 넓은 범위의 단어를 포함하며 생성되는 텍스트의 다양성을 높일 수 있습니다. 그러나 관련성이 낮거나 주제에서 벗어난 콘텐츠가 생성될 위험이 높아질 수 있습니다.

Top-p 값이 낮을수록(0에 가까울수록) 모델이 가능성이 높은 매우 좁은 단어 집합에서 선택하도록 제한하여 보다 단정적이고 집중적인 응답을 유도합니다.

Temperature와 Top P 사용법 비교 (확률 값이 실제 동작 예시는 아님)

입력 값 : "나는 아침에 빵을" ← 확률이 높은 예측 후보를 구함

예측 값 : 먹었다(50%) 구웠다(30%) 샀다(10%) 버렸다(7%) 훔쳤다(3%)

[Temperature = 1.5로 설정했을 때] ← 확률 분포를 평탄하게 함

예측 값 : 먹었다(25%) 구웠다(24%) 샀다(23%) 버렸다(22%) 훔쳤다(6%)

[Top P = 0.5로 설정했을 때] ← 상위 50%의 확률 질량을 차지하는 단어만 선택

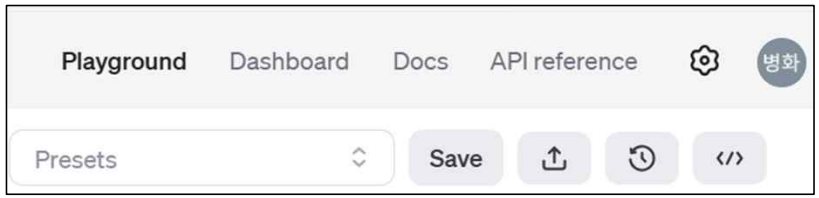
샀다(23%) 버렸다(22%) 훔쳤다(6%)는 배제된다

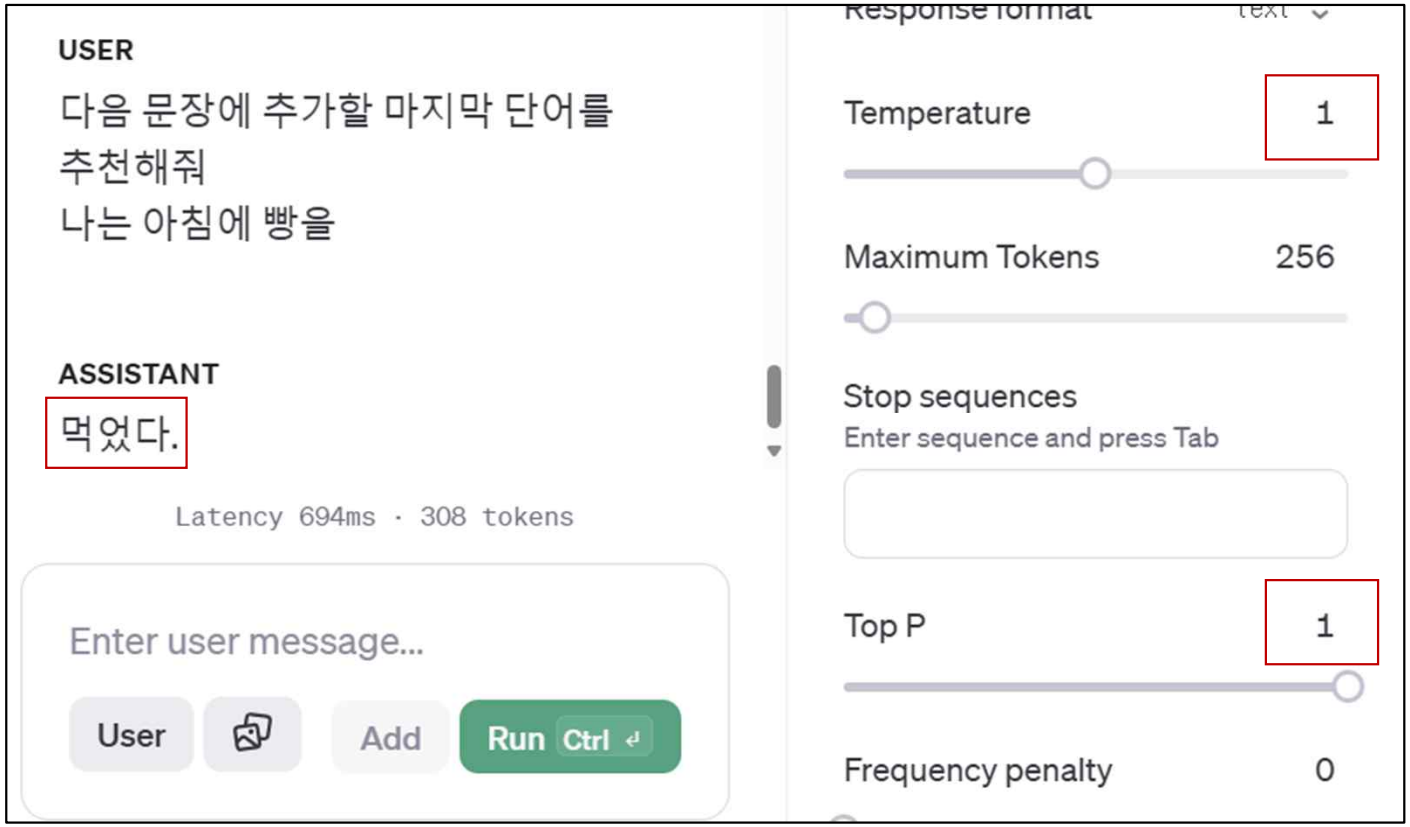
예측 후보 값 : 먹었다(25%) 구웠다(24%) ← 둘 중하나가 랜덤하게 선택(샘플링)

예측 값 : 구웠다 ← 최종 선택

OpenAI Playground 테스트 예시

(Temperature = 1 , Top P = 1)



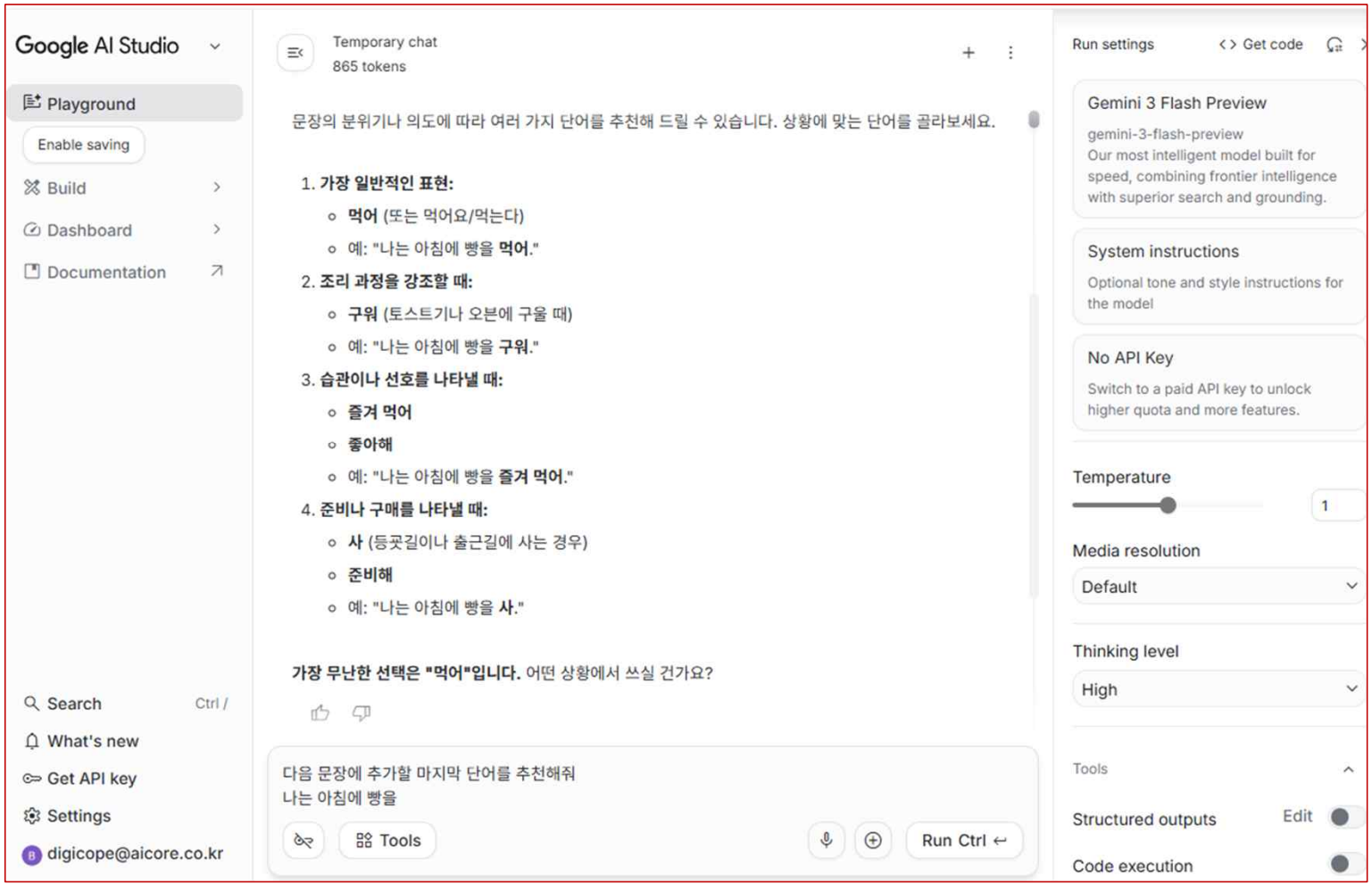


https://platform.openai.com/playground/chat?models=gpt-4o

Google AI Studio Playground 테스트 예시

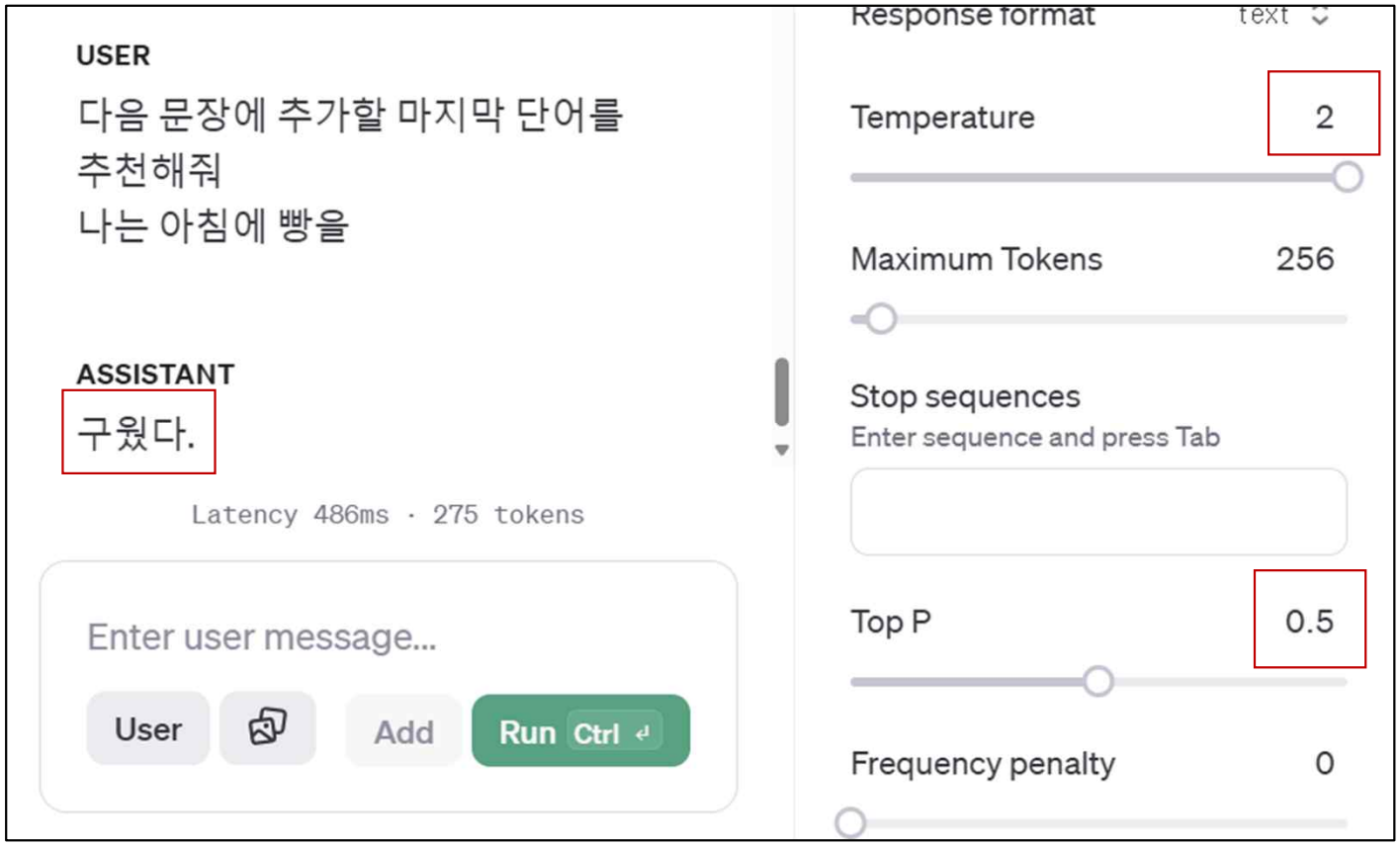
(Temperature = 1)

https://aistudio.google.com/prompts/new_chat



OpenAI Playground 테스트 예시

(Temperature = 2 , Top P = 0.5)

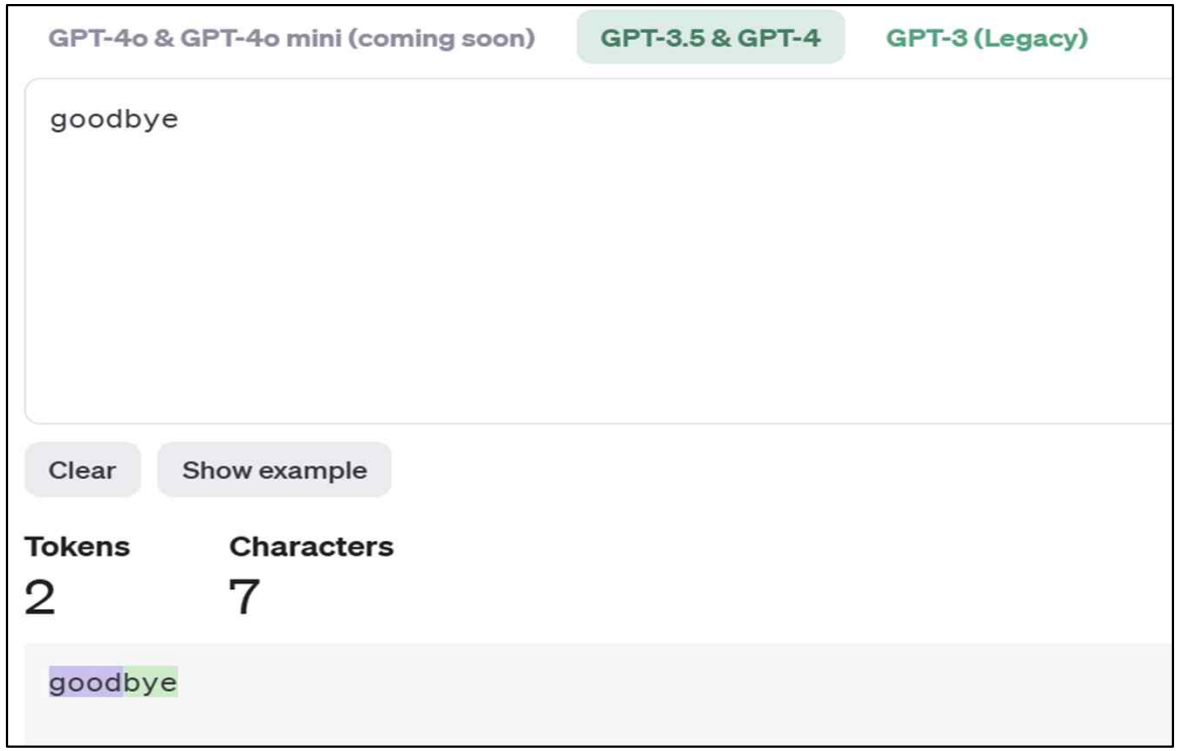


https://platform.openai.com/playground/chat?models=gpt-4o

Google AI Studio Playground 테스트 예시

(Temperature = 2 ,Thinking level = Minimal)

Maximum Token : Token은 단어 개념과 유사, 영어는 보통 1~4글자당 1토큰을 사용, 한글은 1글자당 1~3개 토큰을 사용한다



일반적인 영어 텍스트의 경우 약4자의 텍스트에 해당한다. 대략 단어의 3/4로 변환 된다(100개의 토큰은 약 75단어).

https://platform.openai.com/tokenizer

한글의 토큰 크기 예시

가 : 1 Tokens 토큰 ID: [20565]

가위 : 2 Tokens 토큰 ID: [20565, 82001]

가을 : 2 Tokens 토큰 ID: [20565, 18359]

가격 : 3 Tokens 토큰 ID: [20565, 28740, 102]

생성 : 2 Tokens 토큰 ID: [77535, 33931]

물 : 3 Tokens 토큰 ID: [167, 105, 120]

물건 : 5 Tokens 토큰 ID: [167, 105, 120, 64861, 112]

물총 : 6 Tokens 토큰 ID: [167, 105, 120, 168, 112, 251]

오늘의 날씨 : 9 Tokens 토큰 ID: [58368, 15478, 246, 21028, 38295, 254, 168, 242, 101]

영어의 토큰 크기 예시

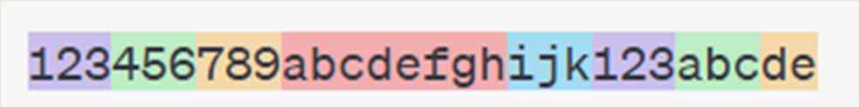
a : 1 Tokens 토큰 ID: [64]

the : 1 Tokens 토큰 ID: [1820]

goodbye : 2 Tokens 토큰 ID: [19045, 29474]

ice cream : 2 Tokens 토큰 ID: [560, 47100]

123456789abcdefghijk123abcde : 8 Tokens 토큰 ID: [4513, 10961, 16474, 57847, 19275, 4513, 13997, 451]



Tom & Jerry : 3 Tokens 토큰 ID: [25763, 612, 29808]

How are you? : 4 Tokens 토큰 ID: [4438, 527, 499, 30]

Nice to meet you. : 5 Tokens 토큰 ID: [46078, 311, 3449, 499, 13]

2. 프롬프트 엔지니어링 기법

2. 프롬프트 엔지니어링 기법

다양한 프롬프트 엔지니어링 기법

(1) Zero-shot Prompting : 모델에게 아무 예시도 주지 않고 바로 문제를 풀게 하는 방식이다. 모델이 처음부터 답을 만들어내야 한다.

(2) Few-shot Prompting : 몇 가지 예시를 제공한 후, 그 예시를 바탕으로 문제를 풀게 하는 방식이다. 예시들이 힌트가 되어 모델이 더 잘 답할 수 있도록 돕는다.

(3) Chain-of-Thought Prompting(CoT): 모델이 답을 내기 전에 문제를 단계별로 생각하도록 유도하는 방식이다. 중간 과정들을 설명하며 답을 도출한다.

2. 프롬프트 엔지니어링 기법

(4) Self-Consistency(SC) : 여러 번의 시도에서 가장 일관된 답을 선택하는 방식이다. 답의 일관성을 높이기 위해 사용된다.

(5) Generate Knowledge Prompting : 필요한 지식을 생성한 다음, 그 지식을 바탕으로 문제를 해결하는 방식이다. 지식 생성과 문제 해결을 분리하는 방식이다.

(6) Prompt Chaining : 여러 개의 프롬프트를 연결해 단계별로 문제를 해결하는 방식이다. 하나의 프롬프트가 다음 단계로 이어진다.

(7) Tree of Thoughts(ToT) : 여러 가지 생각의 가지를 펼치면서 문제를 해결하는 방식이다. 가능한 여러 경로를 탐색하며 답을 찾아간다.

2. 프롬프트 엔지니어링 기법

(8) Retrieval Augmented Generation (RAG) : 외부 데이터베이스에서 정보를 검색해와 답을 생성하는 방식이다. 검색된 정보로 모델이 더 정확한 답을 낼 수 있도록 한다.

(9) Automatic Reasoning and Tool-use : 자동으로 논리를 전개하고, 필요시 도구를 사용해 문제를 푸는 방식이다. 모델이 스스로 툴을 사용해 답을 도출한다.

(10) Automatic Prompt Engineer : 자동으로 적절한 프롬프트를 생성해내는 방식이다. 모델이 최적의 프롬프트를 설계해 문제를 푼다.

(11) Active-Prompt : 프롬프트를 동적으로 변화시키며 문제를 해결하는 방식이다. 상황에 따라 프롬프트를 바꾸면서 답을 찾는다.

2. 프롬프트 엔지니어링 기법

(12) Directional Stimulus Prompting : 특정 방향으로 모델의 답을 유도하는 방식이다. 답이 특정 방향으로 나올 수 있도록 프롬프트를 설계한다.

(13) Program-Aided Language Models : 프로그램의 도움을 받아 문제를 해결하는 방식이다. 코드나 알고리즘이 포함된 프롬프트를 사용한다.

(14) ReAct : 행동과 반응을 결합해 문제를 해결하는 방식이다. 행동(액션)과 그에 대한 피드백을 통해 더 나은 답을 도출한다.

(15) Reflexion : 모델이 스스로의 답을 반성하고 개선하는 방식이다. 답을 낸 후 그 답을 다시 검토해 수정한다.

2. 프롬프트 엔지니어링 기법

(16) Multimodal CoT : 여러 가지 유형의 데이터를 결합해 Chain-of-Thought 방식으로 문제를 푸는 방식이다. 예를 들어, 텍스트와 이미지를 함께 사용한다.

(17) Graph Prompting : 그래프 구조를 사용해 문제를 푸는 방식이다. 그래프 형태로 정보를 조직하고 그 위에서 답을 찾는다.

https://www.promptingguide.ai/kr/techniques

(17) Dynamic Prompting

입력 상황에 따라 프롬프트를 동적으로 생성·변경하여 LLM의 응답 품질을 높이는 기법이다.

정적인 프롬프트(고정 템플릿)와 달리, **사용자 입력·컨텍스트·목표에 맞게 실시간으로 프롬프트를 조합**한다.

1. 정의

Dynamic Prompting은
"프롬프트를 하나로 고정하지 않고, 상황에 맞게 구성 요소를 조합하여 생성하는 방식"이다.

2. 핵심 개념

2.1 구성 요소 기반 프롬프트

프롬프트를 여러 블록으로 나누어 관리한다.

역할(Role)

지침(Instruction)

컨텍스트(Context)

예시(Few-shot)

출력 형식(Output format)

→ 필요에 따라 조합

2.2 입력 기반 변화

사용자 입력에 따라 프롬프트가 바뀐다.

예:

질문 길이 → 상세/요약 모드 변경

도메인 → 전문가 역할 변경

사용자 의도 → 답변 스타일 변경

2.3 조건 기반 분기

조건문처럼 프롬프트를 다르게 구성한다.

```
if 질문 == "코드":
    → 코드 생성 프롬프트
elif 질문 == "개념":
    → 설명형 프롬프트
```

2. 프롬프트 엔지니어링 기법

3. 구조

Dynamic Prompting은 보통 다음 구조를 가진다.
[System Role]
+ [Instruction]
+ [Dynamic Context]
+ [Few-shot Examples (optional)]
+ [User Input]

4. 예시

4.1 정적 프롬프트 (비추천)

"AI를 설명해줘"
→ 항상 동일한 답변

4.2 Dynamic Prompting 적용

사용자 입력 분석 → "초보자 질문"

프롬프트 생성:
"당신은 초보자를 위한 AI 강사입니다.
쉽고 단계적으로 설명하세요.
예시를 포함하세요."
→ 상황 맞춤 응답 생성

2. 프롬프트 엔지니어링 기법

5. 구현 방법

5.1 템플릿 방식

```
template = """
당신은 {role}입니다.
{instruction}

질문:
{user_input}
"""
```

→ 변수 치환 방식

5.2 규칙 기반 (Rule-based)

```
if "코드" in user_input:
    role = "프로그래밍 전문가"
else:
    role = "AI 강사"
```

5.3 RAG 결합

외부 데이터를 동적으로 추가

검색 → 관련 문서 → 프롬프트에 삽입

5.4 Chain 방식 (LangChain 등)

입력 → 분석 → 프롬프트 생성 → 모델 호출

2. 프롬프트 엔지니어링 기법

6. 주요 활용 사례
6.1 챗봇
사용자 수준별 응답
고객 vs 개발자 분기
6.2 코드 생성
언어별 프롬프트 변경
난이도 조절
6.3 교육 시스템
초급 / 중급 / 고급 설명 분리
6.4 RAG 시스템
검색 결과 기반 프롬프트 구성

7. 장점
응답 품질 향상
사용자 맞춤화 가능
다양한 상황 대응
재사용성 높음

8. 단점
설계 복잡
디버깅 어려움
프롬프트 관리 비용 증가

9. 비교

구분	Static Prompt	Dynamic Prompt
구조	고정	가변
유연성	낮음	높음
유지보수	쉬움	복잡
성능	제한적	우수

10. Dynamic Prompt 템플릿 예제

[System]

당신은 상황에 맞게 설명 방식을 바꾸는 AI 전문가이다.

다음 규칙을 반드시 따른다:

1. 사용자 수준 판단
- "쉽게", "처음", "입문" → 초급자
- 그 외 → 일반 사용자

2. 질문 유형 판단
- "코드", "구현" 포함 → 코드 생성
- "정리", "비교", "표" 포함 → 구조화된 설명
- 그 외 → 일반 개념 설명

3. 출력 방식 결정

[초급자일 경우]
- 쉬운 언어 사용
- 단계별 설명
- 예시 포함

2. 프롬프트 엔지니어링 기법

[일반 사용자일 경우]
- 핵심 위주 설명
- 기술 용어 사용 가능

[코드 요청일 경우]
- 코드 + 설명 포함

[표 요청일 경우]
- 표 형태로 정리

[User Question]
{user_input}

[Response]
위 규칙을 모두 적용하여 최적의 답변을 생성하라.

2. 프롬프트 엔지니어링 기법

[동작 예시]

입력 1
"머신러닝 쉽게 설명해줘"

→ 내부 판단
초급자
개념 설명

→ 결과
쉬운 설명 + 예시

1. 공부하는 방식 (비유로 이해하기)

머신러닝은 우리가 시험공부를 하는 과정과 매우 비슷합니다.

- **교과서 준비 (데이터 수집):** 공부할 자료가 필요합니다. 예를 들어 '강아지 사진' 1만 장을 준비합니다.
- **열공하기 (모델 학습):** 컴퓨터가 사진들을 보면서 "강아지는 귀가 이렇게 생겼고, 코는 촉촉하구나!"라며 스스로 특징을 찾아냅니다.
- **시험 보기 (예측/판단):** 이제 처음 보는 사진을 보여주면, 공부한 내용을 바탕으로 "이건 강아지네요!"라고 정답을 맞힙니다.

---

2. 머신러닝의 3가지 공부법

컴퓨터도 학생들처럼 여러 가지 방법으로 공부를 합니다.

공부법	특징	쉬운 예시
지도 학습	문제와 **정답지**를 같이 줌	스팸 메일 샘플을 보고 스팸인지 아닌지 배우기
비지도 학습	정답 없이 **비슷한 것끼리** 묶기	쇼핑몰 고객들을 구매 성향에 따라 그룹 나누기
강화 학습	잘하면 **상**을 주며 반복 학습	알파고가 바둑에서 이기기 위해 최선의 수 찾기

2. 프롬프트 엔지니어링 기법

[동작 예시]

입력 2
"파이썬으로 리스트 정렬 코드 알려줘"
→ 내부 판단
일반 사용자
코드 요청
→ 결과
코드 + 설명

머신러닝 초급 설명 및 예시

파이썬에서는 리스트 자체를 변경하는 `sort()` 메서드와 새로운 리스트를 반환하는 `sorted()` 함수를 주로 사용합니다.

1. 주요 방식 비교 및 구현

방식	메서드/함수	특징	원본 데이터
제자리 정렬	`list.sort()`	리스트 객체 내부에서 직접 정렬 (메모리 절약)	**수정됨**
복사 정렬	`sorted(list)`	정렬된 새로운 리스트를 생성하여 반환	**유지됨**

Sheets로 내보내기

2. 예제 코드

Python

```python
# 샘플 데이터
data =

# 방법 1: 원본을 유지하며 정렬 (sorted)
sorted_data = sorted(data)
print(f"새로운 리스트: {sorted_data}") #
print(f"원본 유지 확인: {data}")        #

# 방법 2: 원본 리스트를 직접 정렬 (sort)
data.sort()
print(f"원본 직접 수정: {data}")        #
```

2. 프롬프트 엔지니어링 기법

[동작 예시]

입력 3
"딥러닝 모델 종류 표로 정리해줘"
→ 내부 판단
구조화 요청
→ 결과
표 형태 출력

사용자님의 요청에 따라 **[일반 사용자용 구조화된 설명]** 방식으로 딥러닝 모델의 주요 종류를 표로 정리해 드립니다.

핵심적인 기술 용어를 포함하여 각 모델의 특징과 주로 사용되는 분야를 분류했습니다.

딥러닝 주요 모델 유형 비교 정리

모델 종류	풀네임	핵심 특징	주요 활용 분야
ANN	Artificial Neural Network	가장 기본적인 인공신경망 구조 (입력-은닉-출력층)	단순 분류 및 회귀, 기 치 예측
CNN	Convolutional Neural Network	이미지의 공간적 정보를 유지하며 특징 추출 (필터 활용)	이미지 인식, 자율주행 료 영상 분석
RNN	Recurrent Neural Network	순차적인 데이터 (Sequence) 처리에 적합, 이전 정보를 기억	번역기, 음성 인식, 주 측, 텍스트 생성
LSTM / GRU	(RNN의 변형)	RNN의 장기 기억 소실 문제를 해결하기 위해 고안됨	긴 문장 번역, 복잡한 열 데이터 분석
GAN	Generative Adversarial Network	생성자와 판별자가 경쟁하며 가짜 데이터를 진짜처럼 생성	딥페이크, 고해상도 이 지 복원, 디자인 생성
Transformer	-	데이터의 관계를 병렬로 처리 (Attention 매커니즘 활용)	**ChatGPT**, BERT, 거 어 모델(LLM)

2. 프롬프트 엔지니어링 기법

이 프롬프트의 핵심은 다음이다.

1. “판단 → 분기 → 적용” 구조
판단: 사용자 입력 분석
분기: 조건별 규칙 선택
적용: 출력 스타일 변경

2. 하나의 프롬프트로 다양한 응답 생성
→ 별도의 여러 프롬프트 필요 없음

이 프롬프트는
→ **“조건 기반으로 역할·스타일·출력을 바꾸는 Dynamic Prompt 템플릿이다.”**

(18) Tool Use

1. 정의

Tool Use란
→ **LLM이 외부 도구(API, DB, 함수 등)를 호출하여 부족한 능력을 보완하는 방식**이다.

2. 왜 필요한가

LLM은 다음 한계를 가진다.
최신 정보 부족
계산 정확도 제한
외부 시스템 접근 불가
→ 이를 해결하기 위해 Tool을 사용한다.

2. 프롬프트 엔지니어링 기법

3. 개념 구조

Tool Use는 다음 흐름으로 동작한다.

사용자 질문
→ LLM 판단
→ Tool 선택
→ Tool 실행
→ 결과 반환
→ LLM이 최종 답변 생성

4. 구성 요소

4.1 LLM (Agent 역할)

어떤 Tool을 쓸지 결정

4.2 Tool

실제 작업 수행

예:
- 검색 API
- 계산기
- DB 조회
- 코드 실행기

4.3 Tool Description

LLM이 Tool을 이해하도록 설명하는 메타정보

5. 예시

5.1 Tool 없이 (문제)

질문: 현재 서울 날씨 알려줘
→ LLM은 정확한 최신 정보 제공 어려움

5.2 Tool 사용

LLM 판단 → "날씨 API 필요"

Tool 호출:
get_weather("Seoul")

결과:
"맑음, 25도"

최종 응답:
"현재 서울은 맑고 25도입니다."

6. 대표 Tool 유형

6.1 검색 Tool
Google Search
Vector DB (RAG)

6.2 계산 Tool
수학 계산기
Python 실행

6.3 외부 API
날씨
금융
지도

6.4 데이터 Tool
BigQuery
SQL DB

8. 실무 구조 (중요)

User Input
→ Prompt 생성 (Dynamic Prompting)
→ LLM 판단 (Tool 필요 여부)
→ Tool 호출
→ 결과 결합
→ 최종 응답

9. 대표 프레임워크

다음 도구들이 Tool Use를 지원한다.
LangChain
OpenAI Function Calling
Google Vertex AI
LlamaIndex

10. 장점
최신 데이터 활용 가능
정확도 향상
복잡한 작업 처리 가능
자동화 가능

11. 단점
설계 복잡
Tool 실패 가능성
비용 증가
latency 증가

감사합니다